

which tells the API when to stop and also ‘flattens’ the coefficients of the inputs. This is done to convert it to a one-dimensional format before being sent for processing in the dense layers. And with just a few finishing touches, the entire program is ready to be compiled and optimised.

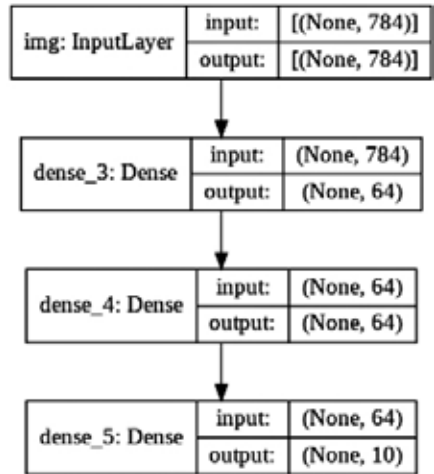
- `model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])`

Keras has a variety of loss functions and user friendly optimisers to choose from that should be selected based on the objective behind the entire program. This can include a tuning algorithm for a random forest model or finding the feature importance list from a neural network schema.

- `model.fit(X_train, Y_train, batch_size=32, nb_epoch=10, verbose=1)`

Once the model has been fit and tested, it has to be imposed against a set of early stopping rules using callbacks. One should save the model weights using the history log or keep it in a separate matrix or dataframe. Finally get the model scores printed and check for accuracies.

- `score = model.evaluate(X_test, Y_test, verbose=0)`



Functional API with layers

System prepping and specifics

If you are planning on having the data inserted into the system using standard json files, it's usually suggested to have the mainframe confirm its packages in the system. Multi-level GPU mining is possible with Keras with some tweak ups. The hardware components used for system prepping and improving neural networks are dependent on the end result. Simple training can be done on normal CPUs while multi-layered testing would require GPUs and even FPGAs.

Most modern programs run on an NVIDIA GPU or an Amazon Web Services C2 GPU instance. Google is also making an entryway into the